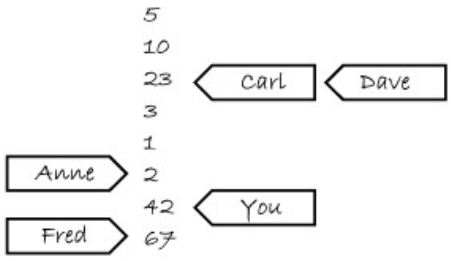
Figure 5.5. The notebook for the man in the cubicle



Now imagine that there's not just one man in a cubicle but a whole cubicle farm, with loads of men with phones and notepads. These are all our atomic variables. Each variable has its own modification order (the list of values on the pad), but there's no relationship between them at all. If each caller (you, Carl, Anne, Dave, and Fred) is a thread, then this is what you get when every operation uses `memory_order_relaxed`. There are a few additional things you can tell the man in the cubicle, such as "write down this number, and tell me what was at the bottom of the list" (`exchange`) and "write down *this* number if the number on the bottom of the list is *that*; otherwise tell me what I should have guessed" (`compare_exchange_strong`), but that doesn't affect the general principle.

If you think about the program logic from listing 5.5, then `write_x_then_y` is like some guy calling up the man in cubicle `x` and telling him to write `true` and then calling up the man in cubicle `y` and telling *him* to write `true`. The thread running `read_y_then_x` repeatedly calls up the man in cubicle `y` asking for a value until he says `true` and then calls the man in cubicle `x` to ask for a value. The man in cubicle `x` is under no obligation to tell you any specific value off his list and is quite within his rights to say `false`.

This makes relaxed atomic operations difficult to deal with. They must be used in combination with atomic operations that feature stronger ordering semantics in order to be useful for inter-thread synchronization. I strongly recommend avoiding relaxed atomic operations unless they're absolutely necessary and even then using them only with extreme caution. Given the unintuitive results that can be achieved with just two threads and two variables in listing 5.5, it's not hard to imagine the possible complexity when more threads and more variables are involved.

One way to achieve additional synchronization without the overhead of full-blown sequential consistency is to use *acquire-release ordering*.

Acquire-Release Ordering

Acquire-release ordering is a step up from relaxed ordering; there's still no total order of operations, but it does introduce some synchronization. Under this ordering model, atomic loads are *acquire* operations (`memory_order_acquire`), atomic stores are *release* operations (`memory_order_release`), and atomic read-modify-write operations (such as `fetch_add()` or `exchange()`) are either *acquire*, *release*, or both (`memory_order_acq_rel`). Synchronization is pairwise, between the thread that does the release and the thread that does the acquire. *A release operation synchronizes-with an acquire operation that reads the value written*. This means that different threads can *still* see different